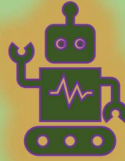


DOMACI Br 2



1. TPC implementacija: TCP je protokol koji garantuje da svaka poruka sigurno stigne od pošiljaoca do primaoca, i to u ispravnom redoslijedu. Prije nego što se ijedna poruka razmijeni, klijent i server moraju se međusobno uskladiti i uspostaviti konekciju. Tek nakon toga komunikacija može početi. U ovom zadatku sam implementirao osnovnu TCP komunikacija između servera i klijenta. Server se pokreće prvi i čeka na portu 1234, a klijent se spaja i šalje poruke koje server prima i vraća nazad. Svaka poruka koja je poslata garantovano je i primljena, što je potvrđeno eksperimentom sa 100 poruka gdje nije zabilježen ni jedan gubitak. Ovaj rezultat se svakako očekuje na localhost mreži:

Server kod:

```
import java.io.*;
import java.net.*;

public class TCPServer {
    public static void main(String[] args) throws IOException {

        ServerSocket serverSocket = new ServerSocket(1234);
        System.out.println("TCP Server pokrenut na portu 1234...");

        Socket clientSocket = serverSocket.accept();
        System.out.println("Klijent se spojio: " + clientSocket.getInetAddress());

        BufferedReader in = new BufferedReader(new
        InputStreamReader(clientSocket.getInputStream()));
        PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);

        String message;
        while ((message = in.readLine()) != null) {
            System.out.println("Primljeno od klijenta: " + message);
            out.println("Server: " + message);
        }

        in.close();
        out.close();
        clientSocket.close();
        serverSocket.close();
    }
}
```

Server se pokreće kreiranjem `ServerSocket` objekta na portu 1234, koji rezerviše taj port i počinje osluškivati dolazne zahtjeve. Linija `serverSocket.accept()` zaustavlja program i čeka dok se klijent ne spoji, tek tada izvršavanje nastavlja dalje. Nakon što se klijent spoji, otvaraju se dvije komunikacione cijevi: `BufferedReader` "in" nam služi za čitanje poruka koje klijent šalje, i `PrintWriter` "out" za slanje odgovora nazad. Petlja `while ((message = in.readLine()) != null)` čita poruke jednu po jednu i za svaku primljenu poruku vraća odgovor sa prefiksom "Server:". Petlja se završava kada klijent zatvori vezu, jer tada `readLine()` vraća `null`. Na kraju se zatvaraju sve cijevi i socketi.

Sa linijom `serverSocket.accept()` sam se igrao kako bih ukapirao kako se koristi kada želimo da ograničimo ulazak klijenta na server. Dva načina na koja sam to uradio su provjera lozinke i filtriranje IP adrese. Provjera lozinke funkcionira tako da server odmah nakon što klijent uspostavi vezu čeka da klijent pošalje lozinku kao prvu poruku. Ako lozinka nije ispravna, server odbija klijenta i zatvara vezu.

Na strani servera dodat je ovaj dio:

```
String sifra = in.readLine();
if (!sifra.equals("tajna123")) {
    out.println("Pristup odbijen.");
    clientSocket.close();
    return;
}
out.println("Pristup dozvoljen.");
```

Na strani klijenta, prije slanja bilo kakve poruke, od korisnika se traži da unese lozinku u terminalu:

```
System.out.print("Unesite sifru za pristup serveru: ");
String sifra = scanner.nextLine();
out.println(sifra);
```

```
String odgovorServera = in.readLine();
System.out.println(odgovorServera);
```

```
if (!odgovorServera.equals("Pristup dozvoljen. ")) {
    System.out.println("Konekcija se zatvara.");
    socket.close();
    return;
}
```

Filtriranje IP adrese radi na sličnom principu tim što odmah nakon `accept()` server čita IP adresu klijenta i provjerava da li je dozvoljena, kao neka vrsta white-liste. U ovom slučaju dozvoljena je samo adresa `127.0.0.1`, što znači samo lokalne (localhost) konekcije:

```
String klijentIP = clientSocket.getInetAddress().getHostAddress();
```

```
if (!klijentIP.equals("127.0.0.1")) {
    out.println("Pristup odbijen - nepoznata adresa.");
    clientSocket.close();
    return;
}
```

Klijent kod:

```
import java.io.*;
import java.net.*;
import java.util.Random;

public class TCPClient {
    public static void main(String[] args) throws IOException {
        Socket socket = new Socket("localhost", 1234);
        System.out.println("Spojen na TCP Server.");
        BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
```

```

String[] poruke = {"Poruka 1: Hej", "Poruka 2: Kako si", "Poruka 3: Sta ima", "Poruka 4: Jesi li
tu", "Poruka 5: Dovidjenja"};
Random random = new Random();
int brojac = 1;
int prim = 0;
for (int i = 0; i < 100; i++) {
    int indeks = random.nextInt(poruke.length);
    out.println(poruke[indeks]);
    String reply = in.readLine();
    if (reply != null) prim++;
    System.out.println("Poslano: poruka broj " + brojac + " " + poruke[indeks]);
    System.out.println("Primljeno: Odgovor " + brojac + " " + reply);
    brojac++;
}
System.out.println("Poslano: 100");
System.out.println("Primljeno: " + prim);
System.out.println("Izgubljeno: " + (100 - prim));
socket.close();
}
}

```

Klijent se spaja na server linijom `new Socket("localhost", 1234)`, `localhost` označava da se server nalazi na istom računaru, a 1234 je port na kojem server čeka (taj port settujemo u Server kodu). Nakon spajanja otvaraju se cijevi za čitanje i pisanje, isto kao na serveru.

U prvoj fazi testiranja, klijent šalje 5 poruka koje su ručno definisane u nizu. Svaka poruka se šalje redom, klijent čeka odgovor servera i ispisuje šta je poslao i šta je primio. Ovim je potvrđeno da osnovna komunikacija između servera i klijenta funkcioniše ispravno.

Nakon što je osnovna implementacija potvrđena, uveo sam dvije izmjene. Petlja je proširena sa 5 što je prvobitno bio samo `length` niza na 100 iteracija kako bi se testirala pouzdanost protokola pri većem broju poruka. Dodao je i `Random` koji u svakoj iteraciji nasumično bira jednu od pet poruka iz niza, što je na neki način načinilo eksperiment realističnijim. Varijabla "prim" sam stavio da broji svaki odgovor koji nije null, tj. sve što prođe, što na kraju omogućava ispis sažetka sa brojem poslatih, primljenih i izgubljenih poruka. U Experimentu, ako sve pođe po planu nebi trebalo da ima ni jedan izgubljen paket jer je sve na localhost.

257	-28.079461000	127.0.0.1	127.0.0.1	TCP	56	51368 → 1234	[SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
258	-28.079357600	127.0.0.1	127.0.0.1	TCP	56	1234 → 51368	[SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
259	-28.079286100	127.0.0.1	127.0.0.1	TCP	44	51368 → 1234	[ACK] Seq=1 Ack=1 Win=65280 Len=0

Na snimku sam izdvojio tri paketa, 257, 258 i 259. Ovo su paketi koji čine TCP handshake, odnosno proces ili trenutak kad se uspostavi konekcija prije nego što ijedna poruka bude razmijenjena. Paket 257 je SYN klijent sa porta 51368 šalje zahtjev serveru na port 1234, što je zapravo isto u kodu(klijenta) gdje je linija `new Socket("localhost", 1234)`. U tom trenutku klijent kuca na vrata servera. Paket 258 je SYN-ACK, tu server odgovara i prihvata zahtjev. Ovo se dešava u trenutku kada server izvršava `serverSocket.accept()` i prestaje blokirati. Paket 259 je ACK, gdje klijent potvrđuje i konekcija je u tom trenutku potpuno uspostavljena. Tek nakon ovog paketa obje strane mogu početi razmjenjivati poruke. Ovo je bitno jer pokazuje da TCP nikad ne počinje slati podatke bez prethodnog dogovora za razliku od UDP-a gdje 'u prikazati da takav proces ne postoji i podaci se šalju odmah.

260	-28.072394100	127.0.0.1	127.0.0.1	TCP	66	51368 → 1234	[PSH, ACK] Seq=1 Ack=1 Win=65280 Len=22
261	-28.072334800	127.0.0.1	127.0.0.1	TCP	44	1234 → 51368	[ACK] Seq=1 Ack=23 Win=65280 Len=0
262	-28.070716500	127.0.0.1	127.0.0.1	TCP	74	1234 → 51368	[PSH, ACK] Seq=1 Ack=23 Win=65280 Len=30
263	-28.070632200	127.0.0.1	127.0.0.1	TCP	44	51368 → 1234	[ACK] Seq=23 Ack=31 Win=65280 Len=0
264	-28.062850600	127.0.0.1	127.0.0.1	TCP	63	51368 → 1234	[PSH, ACK] Seq=23 Ack=31 Win=65280 Len=19
265	-28.062798700	127.0.0.1	127.0.0.1	TCP	44	1234 → 51368	[ACK] Seq=31 Ack=42 Win=65280 Len=0

Ovdje vidimo pakete koji dolaze odmah nakon handshake-a i predstavljaju stvarnu razmjenu poruka između klijenta i servera. Paket 260 je označen sa PSH ACK, to znači da sad klijent šalje poruku serveru, Len=22 znači da paket nosi 22 bajta podataka, što odgovara dužini jedne od poruka koje sam stavio u listi. PSH oznaka znači da podaci trebaju biti odmah prosljeđeni aplikaciji bez čekanja u bufferu. Ovo se u kodu odigrava u liniji `out.println(poruke[indeks])` (dakle u `out`, printuje poruku sa random indeksom). Paket 261 je ACK od servera što znači da server potvrđuje da je primio poruku. Tu se ogleda onaj dio TCP mehanizma pouzdanosti, svaki primljeni paket mora biti potvrđen. Paket 262 je PSH ACK od servera gdje server šalje odgovor nazad klijentu, Len=30 što odgovara dužini odgovora sa prefiksom "Server:" ispred. Ovo odgovara liniji `out.println("Server: " + message)` na serveru (znači opet `out.print` sad sa serverske strane i na klijentsku stranu taj odgovor ide u `in.readLine()`). Paket 263 je ACK od klijenta klijent potvrđuje da je primio odgovor i ciklus se ponavlja za sljedeću poruku i tako do 100 iteracija.

2. UDP implementacija: Za razliku od TCP-a koji prije svake komunikacije mora proći kroz proces uspostavljanja konekcije i potvrđivati svaki primljeni paket, UDP funkcioniše na potpuno drugačijem principu. Svaki paket se šalje samostalno, bez prethodnog dogovora i bez čekanja na potvrdu da je stigao. Upravo zbog toga UDP je bolji izbor u situacijama gdje je brzina važnija od pouzdanosti, kao recimo kod video streaminga ili online gaminga nije problem ako jedan frame ne stigne, dok bi čekanje na potvrdu kao što TCP radi uzrokovalo zamrzavanje slike ili kašnjenje u igrici.

```
import java.net.*;
public class UDPReceiver {
    public static void main(String[] args) throws Exception {
        DatagramSocket socket = new DatagramSocket(9876);
        System.out.println("UDP Receiver pokrenut na portu 9876...");
        byte[] buffer = new byte[1024];
        for (int i = 0; i < 100; i++) {
            DatagramPacket paket = new DatagramPacket(buffer, buffer.length);
            socket.receive(paket);
            String poruka = new String(paket.getData(), 0, paket.getLength());
            System.out.println("Primljeno: " + poruka);
            String odgovor = "ACK: " + poruka;
            byte[] odgovorBytes = odgovor.getBytes();
            DatagramPacket odgovorPaket = new DatagramPacket(
                odgovorBytes,
                odgovorBytes.length,
                paket.getAddress(),
                paket.getPort()
            );
            socket.send(odgovorPaket);
        }
        socket.close();
    }
}
```

Najveća razlika se odmah vidi na početku, postoji samo jedan `DatagramSocket` koji radi sve, i primanje i slanje. Kod TCP-a smo imali `ServerSocket` za slušanje i zasebni `Socket` za komunikaciju, ovdje je to sve jedan objekat. Druga ključna razlika je `byte[] buffer = new byte[1024]`. Kod TCP-a `BufferedReader` je sam upravljao memorijom i davao nam tekst spreman za čitanje. UDP ne radi to automatski, nego moramo ručno da pripremimo praznu kutiju u memoriji prije nego što paket stigne. `DatagramPacket` je koncept koji TCP nema, a ovo u suštini znači da je svaki primljeni paket kao zasebna koverta koja u sebi nosi i podatke i informaciju o tome ko ga je poslao. Upravo zbog toga se iz paketa mora ručno izvući adresa pošiljaoca sa `paket.getAddress()` i `paket.getPort()` kada šaljemo odgovor. Kod TCP-a to nije bilo potrebno jer je konekcija već znala kome da odgovori. Još jedna razlika je pretvaranje podataka, gdje su kod TCP-a `PrintWriter` i `BufferedReader` automatski radili konverziju između teksta i bajtova. Ovdje to radimo ručno: `odgovor.getBytes()` pretvara tekst u bajtove za slanje, a `new String(paket.getData(), 0, paket.getLength())` pretvara primljene bajtove nazad u tekst.

```
import java.net.*;
import java.util.Random;

public class UDPSender {
    public static void main(String[] args) throws Exception {
        DatagramSocket socket = new DatagramSocket();
        InetAddress adresa = InetAddress.getByName("localhost");
        String[] poruke = {"Poruka 1: Hej", "Poruka 2: Kako si", "Poruka 3: Sta ima", "Poruka 4: Jesi li tu", "Poruka 5: Dovidjenja"};
        Random random = new Random();
        int primljeno = 0;
        for (int i = 0; i < 100; i++) {
            int indeks = random.nextInt(poruke.length);
            byte[] podaci = poruke[indeks].getBytes();
            DatagramPacket paket = new DatagramPacket(podaci, podaci.length, adresa, 9876);
            socket.send(paket);
            System.out.println("Poslano: " + poruke[indeks]);
            byte[] buffer = new byte[1024];
            DatagramPacket odgovor = new DatagramPacket(buffer, buffer.length);
            socket.receive(odgovor);
            String reply = new String(odgovor.getData(), 0, odgovor.getLength());
            if (reply != null && !reply.isEmpty()) primljeno++;
            System.out.println("Primljeno: " + reply);
        }
        System.out.println("Poslano: 100");
        System.out.println("Primljeno: " + primljeno);
        System.out.println("Izgubljeno: " + (100 - primljeno));
        socket.close();
    }
}
```

Sender je druga strana komunikacije, dok Receiver čeka pakete na fiksnom portu 9876, Sender ih kreira i šalje. OS mu automatski dodjeljuje slobodan port jer nije bitno da bude dostupan na fiksnoj adresi. Prije samog slanja, Sender mora znati gdje da šalje, a to se radi sa `InetAddress.getByName("localhost")` koji predstavlja adresu Receivera. Svaka poruka se ručno upakuje u `DatagramPacket` zajedno sa tom adresom i portom 9876, i tek tada šalje. Na taj način svaki paket sam zna svoju destinaciju, što je osnova UDP komunikacije. Kao i kod TCP-a, testiranje je krenulo sa 5 poruka kako bih potvrdilo da komunikacija funkcioniše ispravno. Ono što je bilo zanimljivo tokom testiranja je upravo ta priroda UDP-a. Kada sam ugasio Receiver usred slanja, Sender je nastavio slati pakete bez ikakve greške ili upozorenja program je radio normalno kao da se ništa nije desilo. Kod TCP-a u istoj situaciji klijent bi odmah dobio grešku jer TCP stalno prati stanje konekcije i zna kada je druga strana nedostupna.

No.	Time	Source	Destination	Protocol	Length	Info
132	10.819686000	127.0.0.1	127.0.0.1	UDP	52	60665 → 9876 Len=20
133	10.821674300	127.0.0.1	127.0.0.1	UDP	57	9876 → 60665 Len=25
134	10.822407800	127.0.0.1	127.0.0.1	UDP	45	60665 → 9876 Len=13
135	10.822790600	127.0.0.1	127.0.0.1	UDP	50	9876 → 60665 Len=18
136	10.823107200	127.0.0.1	127.0.0.1	UDP	49	60665 → 9876 Len=17
137	10.823342600	127.0.0.1	127.0.0.1	UDP	54	9876 → 60665 Len=22
138	10.823558400	127.0.0.1	127.0.0.1	UDP	52	60665 → 9876 Len=20
139	10.823773800	127.0.0.1	127.0.0.1	UDP	57	9876 → 60665 Len=25
140	10.824089700	127.0.0.1	127.0.0.1	UDP	52	60665 → 9876 Len=20
141	10.824366400	127.0.0.1	127.0.0.1	UDP	57	9876 → 60665 Len=25
142	10.824613100	127.0.0.1	127.0.0.1	UDP	52	60665 → 9876 Len=20

Ovdje se vidi UDP komunikacija filtrirana po portu 9876(što je zapravo port koji sam tazio u receiveru). Za razliku od TCP-a, nema handshake paketa na početku, prva stvar koja se vidi su već podaci koji se razmjenjuju. Paket 132, Sender sa porta 60665 šalje poruku Receiveru na port 9876, Len=20 označava dužinu poruke u bajtovima. Paket 133, Receiver sa porta 9876 odgovara nazad na port 60665, Len=25 jer odgovor sadrži "ACK:" prefiks ispred originalne poruke. Ovo se dešava kroz sve iteracije. Sender šalje, Receiver odgovara, bez ikakvog čekanja ili potvrđivanja između.

3. AI dnevnik

Alat: Claude (Anthropic), claude.ai

Koristio sam Claude (Anthropic)

Kroz razgovor sa Claude-om uspio sam da shvatim stvari koje mi nisu bile jasne, konkretno zašto program staje na liniji `accept()` i čeka klijenta zbog toga sam namjerno ispisao i isprobao dio za situacije za koje treba da se ograniči pristup, takođe mi iz početka nije bila jasna razlika između `ServerSocket` koji samo sluša i `Socket` koji predstavlja stvarnu vezu sa klijentom. Za UDP sam kroz razgovor shvatio zašto mi ručno pravimo datagram-ove a zašto je to kod TCP-a automatski, trebalo mi je dublje objašnjenje bajtova i kako se odražavaju u wireshark log.

Pomogao mi je da testiram situacije kad konekcija nije savršena kako bih pokušao da simuliram stvarni scenarijo. Pomogao mi je takođe kako da filtriram Wireshark log pravom sintaksom.